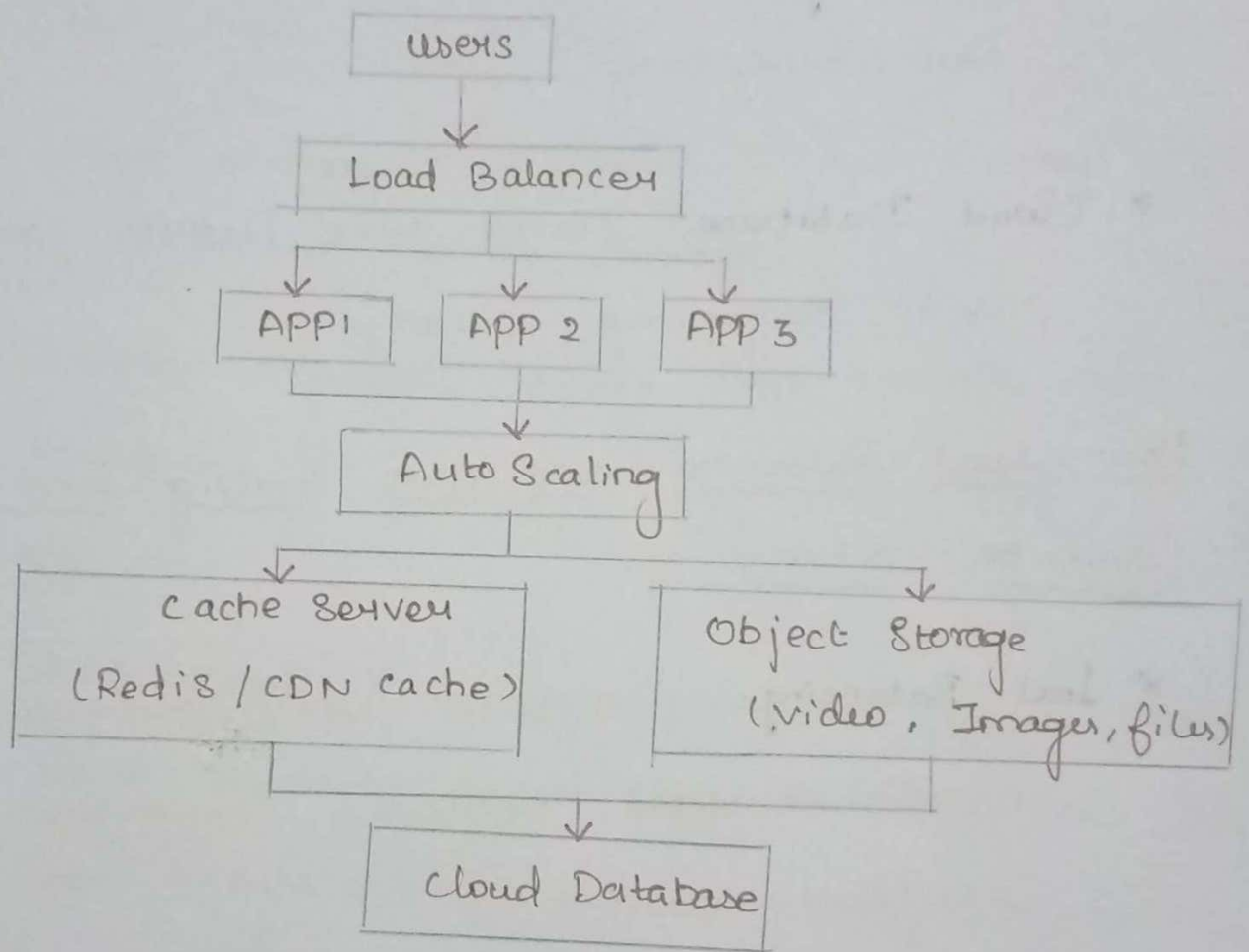


Q1. Design a cloud architecture for a video streaming platform using scalability and caching. Explain the role of load balancing, auto-scaling, and cost optimization.

ARCHITECTURE DIAGRAM



EXPLANATION

- * **Load Balancer**: Distributes incoming user requests evenly among application servers to avoid overload.
- * **Auto Scaling**: Automatically adds servers during high traffic and removes them during low traffic.

* Caching (CDN / Redis): Stores frequently accessed videos near users, reducing buffering and improving streaming speed.

* Cloud Storage: Stores video files securely with high scalability.

* Cloud Database: Stores user details, subscriptions, and streaming history.

How Load Balancing and Auto Scaling Ensure Smooth Streaming.

* Load Balancing: Distributes user requests evenly across multiple servers, preventing server overload.

* Auto Scaling: Automatically adds or removes servers based on traffic, ensuring smooth streaming with minimal buffering.

COST OPTIMIZATION:

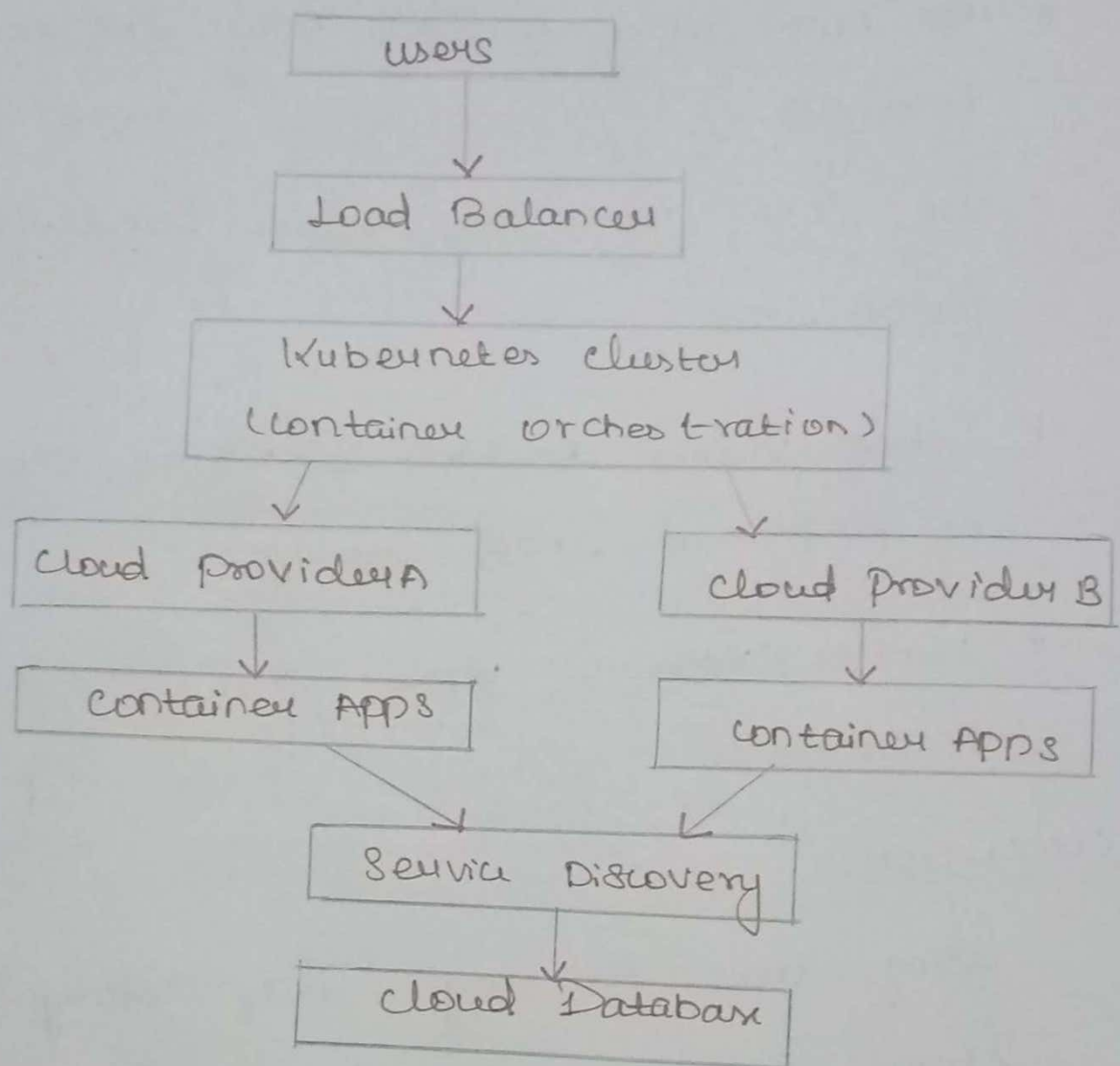
- * Use Auto Scaling to pay only for required resources.
- * Use CDN / cache to reduce bandwidth and server load.
- * Store videos in cloud object storage instead of expensive local storage.
- * Monitor resource usage and remove idle instances.

CONCLUSION:

Using load Balancing, Auto Scaling, caching, cloud storage and a cloud Database provides high availability, minimal buffering, better Performance, cost effective video streaming for millions of users.

Propose a container orchestration and multi-cloud solution. Explain service discovery, scaling, and the benefits and risks of multi-cloud Deployment.

ARCHITECTURE DIAGRAM:



EXPLANATION

* Container orchestration

* Kubernetes manages container deployment, Scaling, and monitoring.

* Ensures applications run consistently across multiple cloud providers.

* Multi-cloud strategy

- * Applications are deployed on two or more cloud providers

- * Improves availability and avoids vendor lock-in

* Service Discovery

- * Automatically finds and connects services within the Kubernetes cluster.

- * Enables smooth communication between containers

* Scaling

- * Kubernetes automatically increases or decreases containers based on workload.

- * Maintains application performance during traffic changes.

BENEFITS

- * High availability

- * Better reliability

- * No vendor lock-in

- * Improved scalability.

RISKS

- * Complex Management
- * Higher operational cost
- * Security and network configuration challenges.

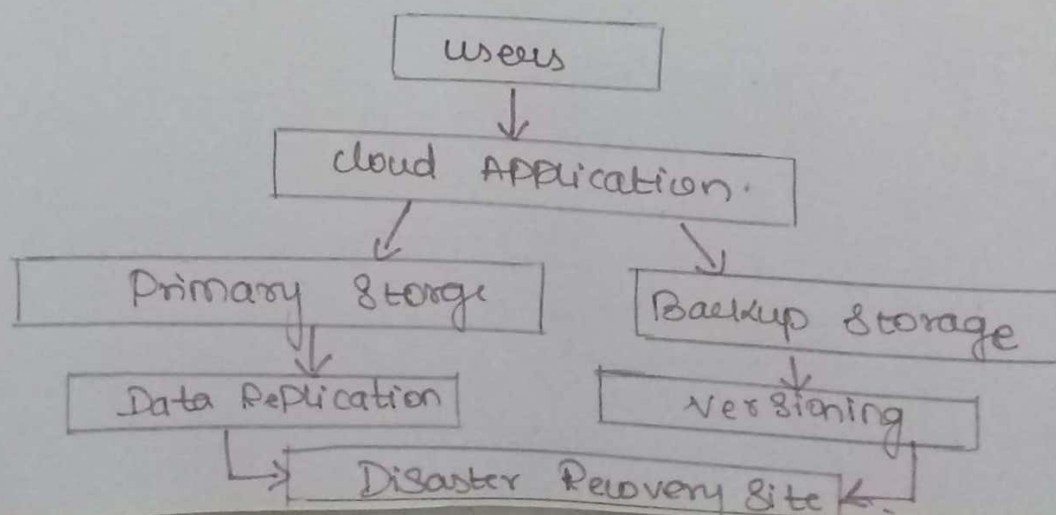
CONCLUSION:

using Kubernetes with a multi-cloud strategy provides reliable deployment, automatic scaling, efficient service discovery, and high availability while reducing dependency on a single cloud provider.

Q3.

Design a cloud backup and disaster recovery solution using backup, replication, and versioning. Explain RTO, RPO, and business continuity.

ARCHITECTURE DIAGRAM:



EXPLANATION

1. cloud Backup

- * Automatically backs up data to cloud storage
- * Protects Data from accidental deletion.

2. Replication

- * Copies data to another location in real time
- * Ensures Data is available if the Primary System fails.

3. Versioning

- * Stores multiple version of files.
- * Allows recovery of deleted or modified files.

4. RTO and RPO

- * RTO (Recovery Time objective): Time required to restore services after a failure.
- * RPO (Recovery Point objective): Maximum acceptable amount of data loss.

5. Business continuity

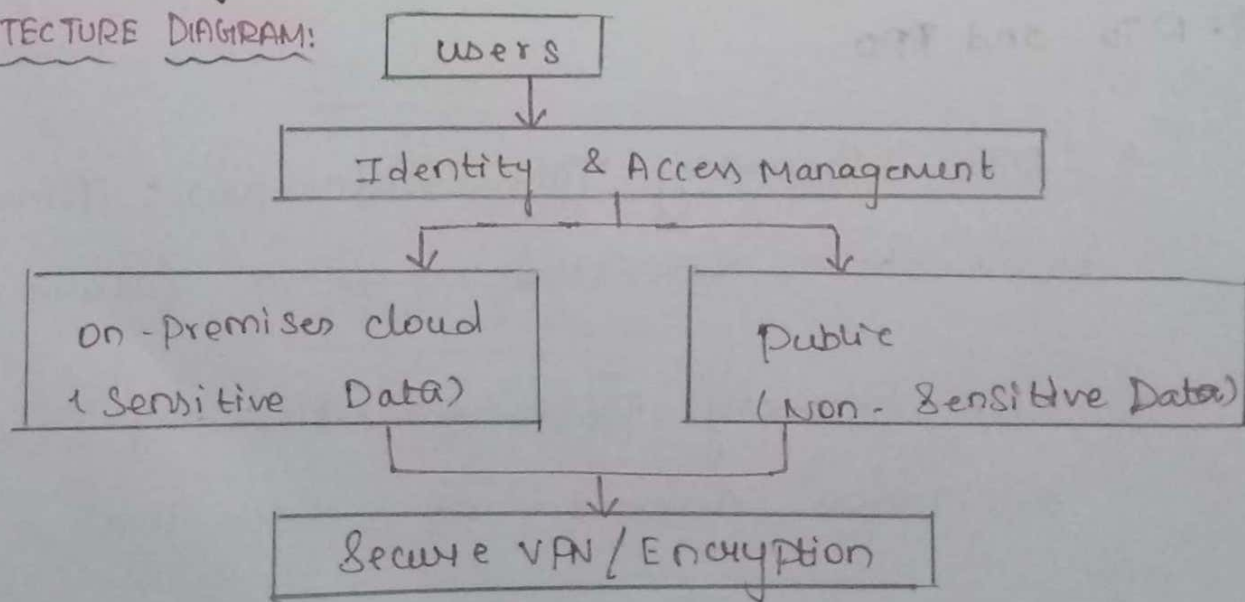
- * Restore data from backups.
- * Switch to the disaster recovery site during failures.
- * Resume services quickly with minimal downtime.

CONCLUSION

Using cloud backup, replication, versioning, and a disaster recovery site ensures data protection, quick recovery, and continuous business operations during failures.

Q4. Design a hybrid cloud architecture for secure communication. Explain identity management, encryption, and challenges in hybrid cloud environments.

ARCHITECTURE DIAGRAM:



1. Hybrid cloud Architecture

- * Store Sensitive data and compliance based application on -Premises Servers.
- * Store non-Sensitive data and applications on the Public cloud.
- * Connect both environments using a Secure VPN or encrypted network for safe communication.

2. Identity Management

- * Implement Identity and Access Management
- * Allow only authorized users to access cloud resources using user authentication and role-based access control.

3. Encryption

- * Encrypt data at rest (stored data) and in transit
- * This protects confidential government data from unauthorized access.

4. challenges

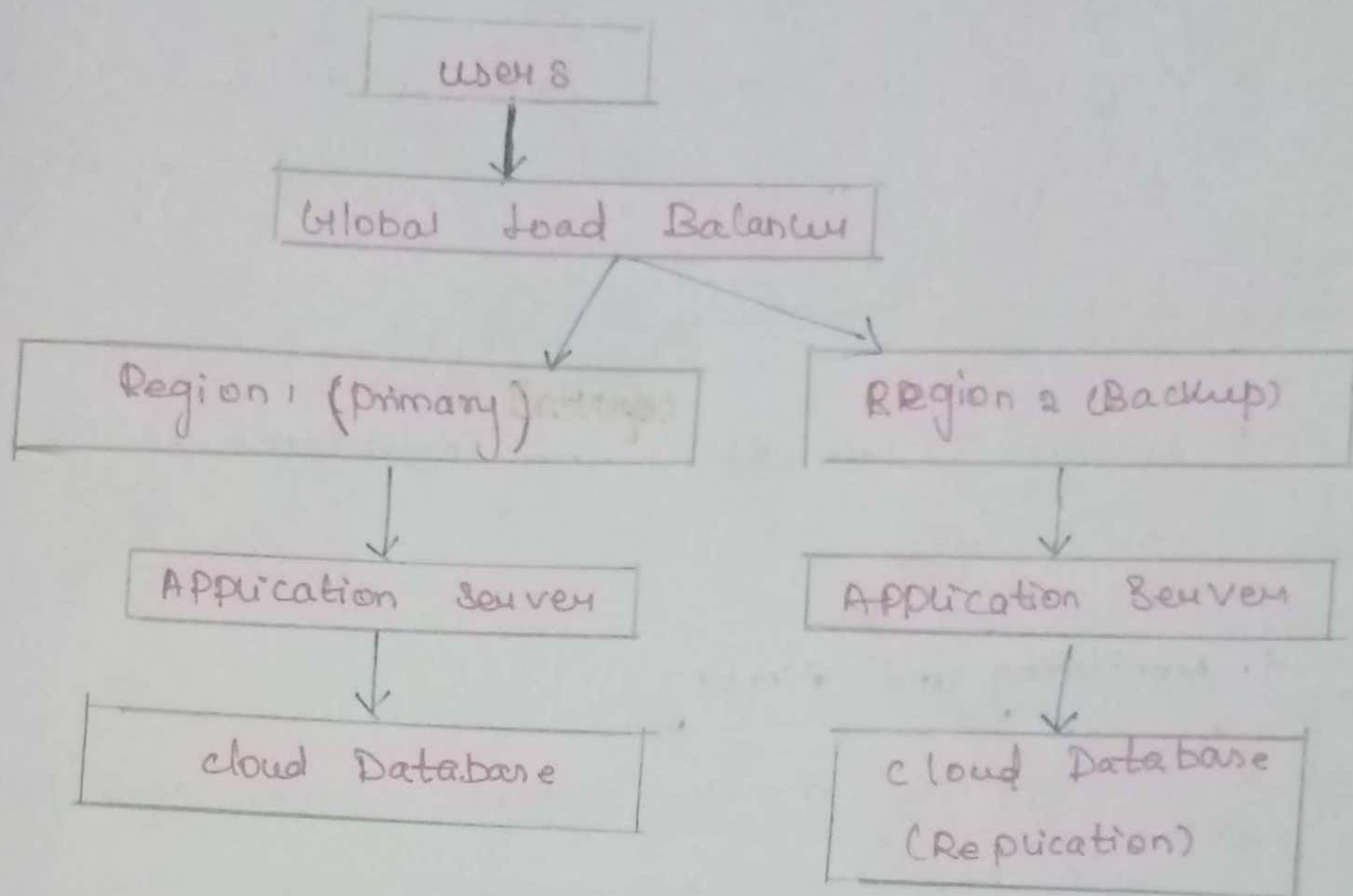
- * Complex management of two environments
- * Maintaining security and regulatory compliance
- * Network latency between on-premises and cloud
- * Higher maintenance and operational costs

CONCLUSION:

A hybrid cloud allows government agencies to securely store sensitive data on-premises while using the public cloud for non-sensitive workloads. IAN, encryption, secure VPN ensure safe communication, while proper management helps overcome hybrid cloud challenges.

Q5.

Propose a multi-region deployment strategy. Explain data replication, failover, monitoring, and alerting.



1. Multi-Region Deployment

- * Deploy the application in multiple cloud regions
- * If the primary regions fails, traffic is automatically redirected to the backup region.

2. Data Replication

- * Replicate data continuously between the Primary and Secondary regions
- * Ensures the latest data is available in both regions

3. Failover Mechanism

- * configure automatic failover to switch users to the backup region when the primary region becomes unavailable.
- * Minimizes downtime and maintains service availability

4. Monitoring and Alerting

- * continuously monitor server health, CPU, memory, and network performance,
- * configure alerts to notify administrators immediately when failures or abnormal activities occur.

CONCLUSION:

A Multi-region deployment with data replication, automatic failover, and continuous monitoring ensures high availability, fault tolerance, uninterrupted service even if one cloud region goes down.